

DL Midsem — Speed Blitz (Exam-Format Answers)

Priority Order — Answers Written As Your Prof Expects Them

Format convention used below, matching what a professor grades for: any *Prove/Show/Derive* question is answered as **Given, To Prove, Proof, Hence Proved**. Any numerical question is answered as **Given, Solution, Answer (boxed)**. *Explain/Differentiate* questions are answered as structured technical writing (definition, then the substantive points, no filler). 47 items, ~5h49m in full, ordered highest-value first — stop wherever you must; everything before that point is the highest-leverage material.

Q1 [4 marks, ~6 min]. Perceptron Convergence Proof

If P, N are finite and linearly separable with margin γ , prove the perceptron algorithm updates its weight vector only finitely many times.

Given: P, N finite and linearly separable, so $\exists w^*$ with $\|w^*\| = 1$ such that $y_i(w^* \cdot x_i) \geq \gamma$ for every point i . Let $R = \max_i \|x_i\|$.

To Prove: The number of weight updates the perceptron algorithm performs is finite.

Proof: Let w_k denote the weight vector after k mistake-driven updates, with $w_0 = 0$. On the k -th mistake, a point (x, y) satisfies $y(w_{k-1} \cdot x) \leq 0$, and the update rule gives $w_k = w_{k-1} + yx$.

Step 1 (lower bound on $w_k \cdot w^$):*

$$w_k \cdot w^* = w_{k-1} \cdot w^* + y(x \cdot w^*) \geq w_{k-1} \cdot w^* + \gamma$$

By induction on k (base case $w_0 \cdot w^* = 0$):

$$w_k \cdot w^* \geq k\gamma$$

Step 2 (upper bound on $\|w_k\|^2$): using $y(w_{k-1} \cdot x) \leq 0$,

$$\|w_k\|^2 = \|w_{k-1}\|^2 + 2y(w_{k-1} \cdot x) + \|x\|^2 \leq \|w_{k-1}\|^2 + R^2$$

By induction on k (base case $\|w_0\|^2 = 0$):

$$\|w_k\|^2 \leq kR^2$$

Step 3 (combine via Cauchy-Schwarz inequality):

$$k\gamma \leq w_k \cdot w^* \leq \|w_k\| \|w^*\| = \|w_k\| \leq \sqrt{kR^2} = \sqrt{k} R$$

$$\Rightarrow k \leq \frac{R^2}{\gamma^2}$$

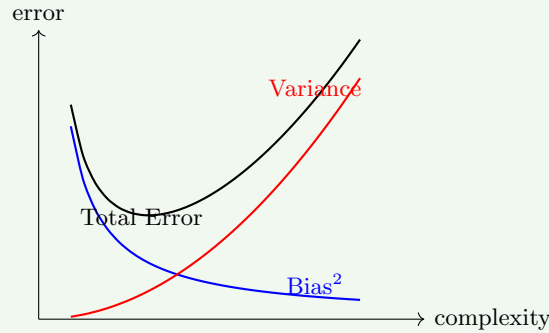
The number of mistakes k is bounded above by the fixed constant R^2/γ^2 , which does not depend on the order in which points are presented. Hence the algorithm can make only finitely many updates before it stops misclassifying points.

Hence Proved. ■

Q2 [5 marks, ~8 min]. Bias-Variance Decomposition

Explain the bias-variance tradeoff with a diagram, and derive predicted error = Bias² + Variance + Noise.

Diagram:



Bias² decreases with model complexity (less underfitting); Variance increases with complexity (more sensitive to the training sample); total expected error is their sum, U-shaped, minimized at intermediate complexity.

Given: $y = f(x) + \varepsilon$, with $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$, and ε independent of the fitted model $\hat{f}$. Let $\bar{f}(x) = \mathbb{E}_D[\hat{f}(x)]$ (expectation over training sets D).

To Prove: $\mathbb{E}[(y - \hat{f}(x))^2] = \text{Bias}^2 + \text{Variance} + \sigma^2$.

Proof:

$$\mathbb{E}[(y - \hat{f})^2] = \mathbb{E}[(f - \bar{f}) + (\bar{f} - \hat{f}) + \varepsilon]^2$$

Expanding the square produces three squared terms and three cross terms. Each cross term vanishes on taking expectation: $(f - \bar{f})$ is a constant (no randomness), $\mathbb{E}[\bar{f} - \hat{f}] = 0$ by the definition of $\bar{f}$, and ε is mean-zero and independent of the model. What remains is:

$$\mathbb{E}[(y - \hat{f})^2] = \underbrace{(\bar{f} - f)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\bar{f} - \hat{f})^2]}_{\text{Variance}} + \underbrace{\mathbb{E}[\varepsilon^2]}_{\sigma^2 \text{ (Noise)}}$$

Hence Proved. ■

Q3 [5 marks, ~8 min]. CNN Output Dimension Formula

Given filter width F , depth d , applied to (W_1, H_1) , derive (W_2, H_2) with stride S , padding P . With K kernels, find the parameter count.

Given: input spatial size (W_1, H_1) , filter size $F \times F \times d$, stride S , padding P , and K filters.

To Derive: the output dimensions (W_2, H_2) and the total parameter count.

Derivation: Padding adds P pixels on each side, so the effective input width becomes $W_1 + 2P$. The filter occupies position 0, and each subsequent valid position advances by S pixels; the

number of positions that fit is

$$\left\lfloor \frac{W_1 + 2P - F}{S} \right\rfloor + 1$$

so, assuming the dimensions divide evenly,

$$W_2 = \frac{W_1 - F + 2P}{S} + 1, \quad H_2 = \frac{H_1 - F + 2P}{S} + 1$$

Each of the K filters has $F \times F \times d$ weights plus 1 bias term, and distinct filters do not share parameters, giving

$$\# \text{params} = K(F^2 d + 1)$$

Q4 [5 marks, ~8 min]. Generic Backprop Gradient

For $x_1, x_2 \rightarrow h$ (activation φ , weights w_1, w_2) $\rightarrow \hat{y}$ (activation φ , weight w_3), $L = \frac{1}{2}(y - \hat{y})^2$: derive $\text{grad}(w_1)$, $\text{grad}(w_2)$, $\text{grad}(w_3)$ for any φ .

Given: forward equations $a_h = w_1 x_1 + w_2 x_2$, $h = \varphi(a_h)$; $a_y = w_3 h$, $\hat{y} = \varphi(a_y)$; loss $L = \frac{1}{2}(y - \hat{y})^2$.

To Derive: $\partial L / \partial w_1$, $\partial L / \partial w_2$, $\partial L / \partial w_3$.

Derivation: By the chain rule, define the output error signal

$$\delta_y = \frac{\partial L}{\partial a_y} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_y} = (\hat{y} - y) \varphi'(a_y)$$

Since $a_y = w_3 h$:

$$\text{grad}(w_3) = \frac{\partial L}{\partial w_3} = \delta_y \cdot \frac{\partial a_y}{\partial w_3} = \delta_y h$$

Propagate the error back through w_3 and the hidden activation to obtain the hidden error signal:

$$\delta_h = \frac{\partial L}{\partial a_h} = \delta_y \cdot \frac{\partial a_y}{\partial h} \cdot \varphi'(a_h) = \delta_y w_3 \varphi'(a_h)$$

Since $a_h = w_1 x_1 + w_2 x_2$:

$$\text{grad}(w_1) = \delta_h x_1 = \delta_y w_3 \varphi'(a_h) x_1, \quad \text{grad}(w_2) = \delta_h x_2 = \delta_y w_3 \varphi'(a_h) x_2$$

These expressions hold for any differentiable φ ; substituting $\varphi'(a) = \varphi(a)(1 - \varphi(a))$ recovers the sigmoid case, and $1 - \tanh^2(a)$ recovers the tanh case.

Q5 [10 marks, ~15 min]. Adam Optimizer — Full Explanation

Explain the Adam optimizer: its components, how it differs from plain SGD, and the specific advantage of using momentum.

Definition. Adam maintains two exponential moving averages of the gradient g_t for each parameter:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{first moment, momentum})$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{second moment, adaptive scale})$$

Since $m_0 = v_0 = 0$, both estimates are biased toward zero in early iterations; this is corrected

by

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

The parameter update is

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Difference from SGD. Plain SGD updates using only the current raw gradient scaled by one fixed global learning rate η . Adam instead uses a direction smoothed by recent gradient history ($\hat{m}_t$) and rescaled per-parameter by that parameter's own recent gradient magnitude ($\hat{v}_t$), adapting the effective step size to each parameter individually.

Advantage of momentum. The first-moment term damps the noisy, oscillating updates that arise from mini-batch gradients, particularly in narrow curved loss valleys where the instantaneous gradient direction fluctuates. It also allows the optimizer to continue moving through small local bumps or saddle-point regions using its accumulated velocity, rather than stalling the moment the instantaneous gradient becomes small.

Remark. With default hyperparameters $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$, when gradients are consistently signed, $\hat{m}_t/\sqrt{\hat{v}_t} \approx \pm 1$, so each step has magnitude close to η regardless of the raw gradient's scale.

Q6 [5 marks, ~8 min]. SGD vs Momentum vs NAG

Differentiate the three.

Stochastic Gradient Descent (SGD): the parameter update uses the gradient computed from a single example or mini-batch, $w_{t+1} = w_t - \eta \nabla f(w_t)$. It is computationally cheap per step but the update direction is noisy.

Momentum GD: an exponentially-weighted running average of past gradients (velocity) is accumulated and used for the update:

$$v_t = \beta v_{t-1} - \eta \nabla f(w_t), \quad w_{t+1} = w_t + v_t$$

This smooths the noise in SGD and accelerates progress in directions where the gradient is consistently signed.

Nesterov Accelerated Gradient (NAG): identical to Momentum, except the gradient used to update the velocity is evaluated at the look-ahead point $w_t + \beta v_{t-1}$ rather than at w_t :

$$v_t = \beta v_{t-1} - \eta \nabla f(w_t + \beta v_{t-1}), \quad w_{t+1} = w_t + v_t$$

This allows NAG to anticipate the effect of the momentum term and decelerate before overshooting a minimum, which plain Momentum cannot do.

Q7 [5 marks, ~8 min]. AdaGrad vs RMSProp

Explain both, with update equations, and how RMSProp fixes AdaGrad's core weakness.

AdaGrad accumulates the sum of squared gradients per parameter and uses it to scale the learning rate:

$$G_t = G_{t-1} + g_t^2, \quad w_{t+1} = w_t - \frac{\eta}{\sqrt{G_t} + \epsilon} g_t$$

Parameters that have historically received large or frequent gradients get a correspondingly

shrunk effective step size, while rarely-updated parameters retain a relatively larger one.

Weakness: G_t is a monotonically increasing sum, so the effective learning rate shrinks toward zero over the course of training and can halt learning well before convergence.

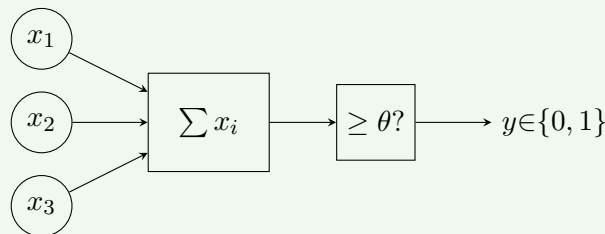
RMSProp replaces the cumulative sum with an exponential moving average of squared gradients:

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta)g_t^2, \quad w_{t+1} = w_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

Because old gradient contributions decay geometrically rather than accumulating forever, the effective learning rate can increase again if recent gradients are small, which resolves AdaGrad's premature decay.

Q8 [3 marks, ~5 min]. McCulloch–Pitts Neuron

Explain the McCulloch–Pitts (MP) neuron and its limitations. How does the perceptron overcome them?



Model. Inputs are binary. Excitatory inputs each carry weight 1 and are summed; the neuron outputs 1 if this sum meets or exceeds a threshold θ , unless any inhibitory input is active, in which case the output is forced to 0 regardless of the excitatory sum.

Limitations: (i) no learning — weights and threshold are fixed by hand for each function; (ii) only Boolean inputs are supported; (iii) only linearly separable Boolean functions can be realized; (iv) all excitatory inputs are forced to equal weight, so inputs cannot be differentially weighted.

How the perceptron overcomes this. The perceptron replaces the fixed weights with learnable, real-valued weights and a bias, adjusted from data via the update rule $w \leftarrow w + \eta(y - \hat{y})x$. This permits real-valued inputs and differential weighting. It remains, however, a linear threshold device, so it still cannot realize non-linearly-separable functions such as XOR.

Q9 [6 marks, ~9 min]. Regularization Methods

Explain L1 regularization, L2 regularization, dropout, and early stopping, and what problem each solves.

L2 regularization (weight decay): adds a penalty $\lambda \|w\|_2^2$ to the loss function. Its gradient contribution $2\lambda w$ shrinks every weight smoothly toward, but not exactly to, zero, reducing the model's sensitivity to any single feature and thereby lowering variance.

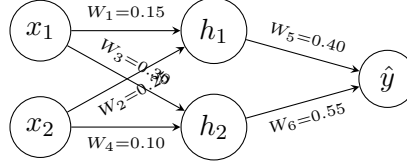
L1 regularization: adds a penalty $\lambda \|w\|_1$. Unlike L2, this drives many weights exactly to zero, producing a sparse weight vector that performs implicit feature selection.

Dropout: during training, each neuron's output is independently zeroed with probability $1 - p$ on every forward pass. This prevents neurons from co-adapting to compensate for one another's errors, and approximates training an ensemble of thinner sub-networks that share weights.

Early stopping: training is monitored via a held-out validation set; training is halted once validation loss begins increasing even as training loss continues to decrease. This stops training at the point the model begins fitting noise specific to the training set rather than the underlying pattern.

Q10 [8 marks, ~16 min]. Full Forward + Backward Pass (Numeric)

$x_1 = 0.3, x_2 = 0.7$; sigmoid hidden h_1, h_2 , sigmoid output $\hat{y}$.



($b_1=0.05, b_2=0.10, b_o=0.05$.) Target $y = 0.8$, $\eta = 0.3$, loss is MSE. Carry out the full forward pass and then the full backward pass, updating every weight and bias.

Given: the network above, $y = 0.8$, $\eta = 0.3$, $L = \frac{1}{2}(y - \hat{y})^2$.

Solution.

Forward pass:

$$h_1 = \sigma(0.15(0.3) + 0.25(0.7) + 0.05) = \sigma(0.27) = 0.5671$$

$$h_2 = \sigma(0.30(0.3) + 0.10(0.7) + 0.10) = \sigma(0.26) = 0.5646$$

$$\hat{y} = \sigma(0.40(0.5671) + 0.55(0.5646) + 0.05) = \sigma(0.5874) = 0.6428$$

Backward pass. Output error signal:

$$\delta_{\text{out}} = (\hat{y} - y) \hat{y}(1 - \hat{y}) = (-0.1572)(0.2296) = -0.03610$$

Output layer updates:

$$W_5^{\text{new}} = 0.40 - 0.3(\delta_{\text{out}} h_1) = 0.40 - 0.3(-0.02047) = 0.40614$$

$$W_6^{\text{new}} = 0.55 - 0.3(\delta_{\text{out}} h_2) = 0.55611, \quad b_o^{\text{new}} = 0.05 - 0.3\delta_{\text{out}} = 0.06083$$

Hidden error signals:

$$\delta_{h_1} = \delta_{\text{out}} W_5 h_1 (1 - h_1) = -0.003545, \quad \delta_{h_2} = \delta_{\text{out}} W_6 h_2 (1 - h_2) = -0.004881$$

Hidden layer updates:

$$W_1^{\text{new}} = 0.15032, \quad W_2^{\text{new}} = 0.25074, \quad b_1^{\text{new}} = 0.05106$$

$$W_3^{\text{new}} = 0.30044, \quad W_4^{\text{new}} = 0.10103, \quad b_2^{\text{new}} = 0.10146$$

Answer:

$$W_1=0.15032, W_2=0.25074, W_3=0.30044, W_4=0.10103, W_5=0.40614, W_6=0.55611$$

$$b_1=0.05106, b_2=0.10146, b_o=0.06083$$

Q11 [6 marks, ~12 min]. MSE vs BCE (Numeric)

Spam classifier. (i) Why is MSE poor here? (ii) BCE formula. (iii) Loss for $y=0, \hat{y}=0.95$. (iv) Average J for $Y=(1, 0, 1, 0)$, $\hat{Y}=(0.9, 0.1, 0.6, 0.4)$.

(i) With a sigmoid output, the MSE gradient contains a factor $\hat{y}(1 - \hat{y})$ which tends to zero as $\hat{y}$ approaches a confidently wrong prediction, so learning stalls exactly when correction is most needed.

(ii) $L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$.

(iii) Given: $y = 0$, $\hat{y} = 0.95$. Solution: $L = -\log(1 - 0.95) = -\log(0.05)$. Answer: $L = 2.996$

(iv) Given: $Y = (1, 0, 1, 0)$, $\hat{Y} = (0.9, 0.1, 0.6, 0.4)$. Solution:

$$y=1, \hat{y}=0.9 : -\log(0.9) = 0.1054; \quad y=0, \hat{y}=0.1 : -\log(0.9) = 0.1054$$

$$y=1, \hat{y}=0.6 : -\log(0.6) = 0.5108; \quad y=0, \hat{y}=0.4 : -\log(0.6) = 0.5108$$

$$J = \frac{0.1054 + 0.1054 + 0.5108 + 0.5108}{4}$$

Answer: $J = 0.3081$

Q12 [5 marks, ~10 min]. Convolution + Max Pooling (Numeric)

5×5 input $\begin{pmatrix} 3 & 1 & 2 & 0 & 1 \\ 0 & 2 & 1 & 3 & 2 \\ 1 & 0 & 3 & 1 & 0 \\ 2 & 1 & 0 & 2 & 3 \\ 0 & 3 & 1 & 0 & 2 \end{pmatrix}$, filter $\begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}$, stride 1, no padding. Find the convolution out-

put, apply 2×2 max pooling (stride 1), and state the padding needed to keep the convolution output the same size as the input.

Given: the input and filter above, stride 1, no padding.

Solution. Output size: $(5-3)/1+1 = 3$, so the convolution output is 3×3 . Sample computation at the top-left position (rows 0-2, cols 0-2):

$$\begin{pmatrix} 3 & 1 & 2 \\ 0 & 2 & 1 \\ 1 & 0 & 3 \end{pmatrix} \cdot \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix} = (-3+0+2) + (0+0+1) + (-1+0+3) = 2$$

Repeating this at all 9 positions gives:

$$\text{Conv output} = \begin{pmatrix} 2 & 1 & -3 \\ 1 & 3 & 1 \\ 1 & -1 & 1 \end{pmatrix}$$

For 2×2 max pooling with stride 1, the output size is $(3-2)/1+1 = 2$. Each of the four 2×2 windows in this 3×3 grid contains the single center cell (value 3, which is also the overall maximum), so:

$$\text{Max-pooled output} = \begin{pmatrix} 3 & 3 \\ 3 & 3 \end{pmatrix}$$

Padding for a same-size convolution output: $P = (F-1)/2 = (3-1)/2 = 1$.

Answer: Conv output $\begin{pmatrix} 2 & 1 & -3 \\ 1 & 3 & 1 \\ 1 & -1 & 1 \end{pmatrix}$; max-pooled $\begin{pmatrix} 3 & 3 \\ 3 & 3 \end{pmatrix}$; padding $P = 1$.

Q13 [4 marks, ~8 min]. CNN Dims + Params, Stride/Padding (Numeric)

A $7 \times 7 \times 1$ input is convolved with $K = 8$ filters of size 3×3 , stride $S = 2$, padding $P = 1$. Find the output size and parameter count.

Given: $W_1 = 7, F = 3, S = 2, P = 1, K = 8, d = 1$.

Solution:

$$W_2 = \frac{7 - 3 + 2(1)}{2} + 1 = \frac{6}{2} + 1 = 4$$

$$\#params = K(F^2 d + 1) = 8(3^2(1) + 1) = 8(10) = 80$$

Answer: $\boxed{\text{output } 4 \times 4 \times 8, \#params = 80}$

Q14 [5 marks, ~10 min]. Full CNN Architecture (Numeric)

INPUT($64 \times 64 \times 3$) $\rightarrow$ CONV-5-16 $\rightarrow$ POOL-2 $\rightarrow$ CONV-3-32 $\rightarrow$ POOL-2 $\rightarrow$ FC-10 (stride 1, no padding on convs). Compute the output size and parameter count at every layer.

Given: the architecture above; CONV- F - K = filter $F \times F$, K filters, stride 1, no padding; POOL-2 = 2×2 , stride 2.

Solution:

Layer	Output	Parameters
INPUT	$64 \times 64 \times 3$	—
CONV-5-16	$60 \times 60 \times 16$	$16(5^2(3) + 1) = 1216$
POOL-2	$30 \times 30 \times 16$	0
CONV-3-32	$28 \times 28 \times 32$	$32(3^2(16) + 1) = 4640$
POOL-2	$14 \times 14 \times 32$	0
FC-10	$6272 \rightarrow 10$	$6272(10) + 10 = 62730$

Answer: Total parameters = $1216 + 4640 + 62730 = \boxed{68,586}$

Q15 [5 marks, ~10 min]. Perceptron Hand-Trace (Numeric)

$\eta=1$, $w_0=(0,0)$, a point with $w \cdot x \leq 0$ counts as misclassified. $x_1=(1,0), y=+1$; $x_2=(0,1), y=+1$; $x_3=(-1,0), y=-1$; $x_4=(0,-1), y=-1$. Trace to convergence.

Given: the dataset and initialization above, prediction rule $\hat{y} = +1$ iff $w \cdot x > 0$.

Solution. Pass 1: x_1 : $w \cdot x = 0 \Rightarrow$ mistake, update $w = (1,0)$. x_2 : $w \cdot x = 0 \Rightarrow$ mistake, update $w = (1,1)$. x_3 : $w \cdot x = -1 < 0$, correct, no update. x_4 : $w \cdot x = -1 < 0$, correct, no update. Pass 2: all four points are now correctly classified $\Rightarrow$ the algorithm has converged.

Answer: $\boxed{w_{\text{final}} = (1,1)}$, reached after 2 updates.

Q16 [4 marks, ~6 min]. Softmax vs Sigmoid

What is softmax? How does it differ from sigmoid, and when is each used?

Definition. For a vector of scores z , $\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$, producing a probability distribution over K classes whose entries sum to 1; increasing one entry's score necessarily decreases the others' share.

Difference from sigmoid. Sigmoid maps each unit to an independent value in $(0, 1)$ with no cross-normalization between units.

When to use each. Softmax is used for single-label multi-class classification, where exactly one class is correct and the classes compete for probability mass. Sigmoid is used for binary classification, or multi-label classification where an instance may belong to several classes simultaneously and each is judged independently.

Q17 [4 marks, ~6 min]. Activation Functions + Sigmoid Derivative

Why are activation functions needed? What are the characteristics of a good one? Explain sigmoid's merits and drawbacks via its derivative.

Necessity. Without a non-linear activation, a composition of linear layers reduces algebraically to a single linear layer, so a network could only represent linear functions regardless of depth. Non-linear activations are what allow a deep network to represent non-linear functions.

Characteristics of a good activation function: differentiable almost everywhere; non-saturating over most of its useful input range (so gradients do not vanish); ideally zero-centered; computationally inexpensive.

Sigmoid. $\sigma(z) = \frac{1}{1 + e^{-z}}$, with derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, obtained by differentiating $(1 + e^{-z})^{-1}$ via the chain rule. *Merits:* smooth and bounded, output interpretable as a probability. *Drawbacks:* saturates for large $|z|$, causing vanishing gradients that compound across layers; output is not zero-centered; the exponential is computationally costly.

Q18 [4 marks, ~6 min]. True/False: Bias-Variance + NAG

(a) Simple models have high bias and high variance. (b) High bias and low variance are desirable. (c) Variance contributes to error, bias does not. (d) In NAG, the gradient term is added to the old weight first, then the momentum term.

(a) **False.** Simple models have high bias but *low* variance, since they are insensitive to variation in the training data.

(b) **False.** The desirable combination is low bias *and* low variance; high bias alone indicates systematic underfitting regardless of variance.

(c) **False.** Both contribute to expected error: $\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Noise}$.

(d) **False.** In NAG, the momentum term is applied first to obtain the look-ahead position $w_t + \beta v_{t-1}$, and the gradient is then computed and added at that look-ahead point.

Q19 [3 marks, ~5 min]. Adam Bias Correction

Show why the bias-correction factor is necessary, and show it disappears as $t \rightarrow \infty$.

Given: $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$, $m_0 = 0$.

To Show: the correction $\hat{m}_t = m_t/(1 - \beta_1^t)$ is necessary, and the correction factor $\rightarrow 1$ as $t \rightarrow \infty$.

Proof. Expanding the recursion:

$$m_t = (1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} g_i$$

If every past gradient equalled the true gradient g , this sums to

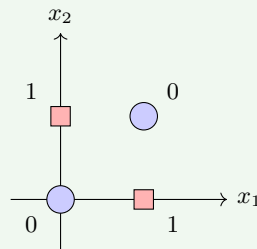
$$m_t = (1 - \beta_1)g \sum_{i=1}^t \beta_1^{t-i} = (1 - \beta_1)g \cdot \frac{1 - \beta_1^t}{1 - \beta_1} = g(1 - \beta_1^t)$$

so m_t underestimates g by the factor $(1 - \beta_1^t)$, an effect that is largest at small t (since $m_0 = 0$). Dividing by exactly this factor, $\hat{m}_t = m_t/(1 - \beta_1^t)$, recovers an unbiased estimate. As $t \rightarrow \infty$, $0 < \beta_1 < 1$ implies $\beta_1^t \rightarrow 0$, so $(1 - \beta_1^t) \rightarrow 1$ and the correction factor approaches 1, becoming unnecessary. The identical argument applies to v_t with β_2 .

Hence Shown. ■

Q20 [3 marks, ~5 min]. XOR Cannot Be Solved by a Single Perceptron

Prove this.



To Prove: no single perceptron can realize the XOR function.

Proof (by contradiction). As the diagram shows, no line separates $\{(0,0), (1,1)\}$ (output 0) from $\{(0,1), (1,0)\}$ (output 1). Suppose weights w_1, w_2, b existed such that

$$\hat{y} = 1 \iff w_1 x_1 + w_2 x_2 + b \geq 0$$

realizes XOR. Then the four training points require:

$$b < 0 \quad (\text{from } (0,0)), \quad w_1 + w_2 + b < 0 \quad (\text{from } (1,1))$$

$$w_2 + b \geq 0 \quad (\text{from } (0,1)), \quad w_1 + b \geq 0 \quad (\text{from } (1,0))$$

Adding the last two inequalities:

$$w_1 + w_2 + 2b \geq 0 \Rightarrow w_1 + w_2 \geq -2b$$

Combined with $w_1 + w_2 < -b$ (from the second condition):

$$-2b \leq w_1 + w_2 < -b \Rightarrow -2b < -b \Rightarrow b > 0$$

This contradicts $b < 0$ established above. Hence no such w_1, w_2, b exist.

Hence Proved. ■

Q21 [3 marks, ~5 min]. Bias-Variance Tradeoff — Concept

Explain the bias-variance tradeoff conceptually.

Bias is the error component arising from a model too simple to capture the true underlying pattern; it is systematically wrong regardless of which training set is used, producing **underfitting**.

Variance is the error component arising from a model too sensitive to the specific training data it happened to see; different training sets yield substantially different fitted models, producing **overfitting**.

Because increasing model complexity typically decreases bias while increasing variance, the two cannot generally be minimized simultaneously — see the diagram in Q2, where total expected error is minimized at an intermediate complexity.

Q22 [5 marks, ~8 min]. CNN Concepts

Explain the convolution operation, stride, padding, pooling, and the effect of extreme filter sizes ($F = 1$ vs. $F = n$).

Convolution. A filter slides across the input; at each position, the dot product of the filter with the covered patch produces one value of the output feature map. This is governed by locality (nearby pixels determine local structure) and weight sharing (the same filter is reused at every position, which is why convolutional layers have far fewer parameters than a fully-connected layer over the same input).

Stride is the number of pixels the filter advances between positions; a larger stride visits fewer positions, producing a smaller output and retaining less spatial detail.

Padding adds a border (typically zeros) around the input so that border pixels are covered by as many filter positions as interior pixels, and to control the output size; "valid" convolution uses no padding (output shrinks), "same" convolution pads enough that output size equals input size.

Pooling (e.g. max pooling) summarizes each small window to a single value; it introduces no learnable parameters, reduces computation, and provides mild translation invariance.

$F = 1$: the output retains the input's spatial size, and each output value is a linear recombination across channels only, with no spatial mixing.

$F = n$ (filter as large as the entire input, no padding): only one valid filter position exists, so the output collapses to a single value per filter — equivalent to a fully-connected layer applied to the entire input, discarding all spatial structure.

Q23 [5 marks, ~8 min]. Xavier Initialization Derivation

Derive the Xavier/Glorot initialization variance from the requirement that a linear layer's output variance equals its input variance.

Given: a linear layer $y = \sum_{i=1}^n w_i x_i$, with w_i, x_i mutually independent, zero-mean random variables, identically distributed across i .

To Derive: $\text{Var}(w)$ such that $\text{Var}(y) = \text{Var}(x)$.

Derivation:

$$\text{Var}(y) = \text{Var}\left(\sum_{i=1}^n w_i x_i\right) = \sum_{i=1}^n \text{Var}(w_i x_i) = \sum_{i=1}^n \text{Var}(w_i) \text{Var}(x_i) = n \text{Var}(w) \text{Var}(x)$$

using the identity $\text{Var}(AB) = \text{Var}(A)\text{Var}(B)$, valid for independent zero-mean A, B . Setting $\text{Var}(y) = \text{Var}(x)$:

$$n \text{Var}(w) = 1 \Rightarrow \boxed{\text{Var}(w) = \frac{1}{n}}$$

An identical argument applied to the backward pass (gradients propagating through w^T) gives $\text{Var}(w) = 1/n_{out}$. The Xavier/Glorot initialization balances both requirements by averaging:

$$\text{Var}(w) = \frac{2}{n_{in} + n_{out}}$$

Q24 [3 marks, ~5 min]. He Initialization Reasoning

Explain why He initialization uses $2/n_{in}$ instead of Xavier's $1/n_{in}$.

ReLU sets to zero every neuron whose pre-activation is negative, which on average zeroes roughly half the units in a layer. If Xavier's variance $1/n_{in}$ (derived assuming the activation preserves variance symmetrically, as is roughly true for sigmoid/tanh near zero) were used with ReLU, the forward-propagated signal variance would shrink by approximately half at every layer, compounding into vanishing activations in a deep network. He initialization doubles the variance to $2/n_{in}$ to exactly compensate for this expected halving, keeping signal variance stable across layers.

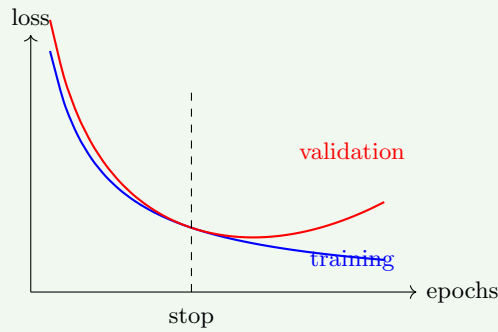
Q25 [2 marks, ~3 min]. Step Activation with Adam

A non-linearity is defined as $f(x) = 4$ if $x \geq 0$, else -2 . Would you use this in a deep network trained with Adam?

No. This is a step function; its derivative is 0 everywhere except at an undefined discontinuity at $x = 0$. Consequently every weight upstream of a neuron using this activation receives a gradient of 0 almost everywhere. Adam only rescales an existing gradient signal via momentum and adaptive per-parameter scaling; it cannot generate a gradient where none exists. No learning would occur through this activation, regardless of which optimizer is used.

Q26 [2 marks, ~3 min]. Early Stopping Diagram

Describe the early-stopping diagram: axes, curves, and the stopping point.



x-axis: training epochs/iterations. **y-axis:** loss. **Training loss curve:** monotonically decreasing throughout training. **Validation loss curve:** decreases initially, reaches a minimum, then increases as the model begins to overfit. **Stopping point:** exactly at the minimum of the validation loss curve, even though the training loss would continue to decrease if training were allowed to continue.

Q27 [2 marks, ~3 min]. 8 Forms of Regularization

List at least 8 forms of regularization used in deep learning.

L1 regularization; L2 regularization (weight decay); dropout; early stopping; dataset augmentation; batch normalization (regularizing side-effect); noise injection into inputs or weights; parameter sharing/tying; ensembling/bagging.

Q28 [2 marks, ~3 min]. Classification Loss Function

Which loss function is used for a classification problem? Define it mathematically.

For binary classification, **binary cross-entropy** is used:

$$L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

For multi-class classification, **categorical cross-entropy** is used:

$$L = - \sum_k y_k \log \hat{y}_k$$

Q29 [2 marks, ~3 min]. Neural Network Parameter Count Derivation

A 1-hidden-layer network has n inputs, k hidden neurons, m outputs, with bias at every neuron. Derive the total parameter count.

Derivation: each of the k hidden neurons receives n inputs and 1 bias, contributing $k(n + 1)$ parameters. Each of the m output neurons receives k inputs and 1 bias, contributing $m(k + 1)$ parameters. These two layers share no parameters, so

$$\boxed{\# \text{params} = k(n + 1) + m(k + 1)}$$

Q30 [2 marks, ~3 min]. XOR-3 Minimum Units

Derive the minimum number of perceptron units sufficient to realize 3-variable XOR (parity) in a single hidden layer, and give the construction.

Construction: let $s = x_1 + x_2 + x_3$; the parity function equals 1 exactly when s is odd. Define three threshold hidden units:

$$h_1 = \mathbb{K}[s \geq 1], \quad h_2 = \mathbb{K}[s \geq 2], \quad h_3 = \mathbb{K}[s \geq 3]$$

(each a perceptron with weights 1, 1, 1 and thresholds 0.5, 1.5, 2.5 respectively). The output unit computes $\text{sign}(h_1 - h_2 + h_3 - 0.5)$:

s	h_1	h_2	h_3	$h_1 - h_2 + h_3$
0	0	0	0	0
1	1	0	0	1
2	1	1	0	0
3	1	1	1	1

The output is 1 exactly when s is odd, correctly realizing parity.

Answer: 3 hidden units suffice.

Q31 [2 marks, ~4 min]. MP Neuron: 3-Bit AND (Numeric)

Design an MP neuron to model a 3-bit logical AND.

Solution: use three excitatory inputs, each of weight 1, with threshold $\theta = 3$:

$$y = 1 \iff x_1 + x_2 + x_3 \geq 3$$

Answer: this is 1 only when all three inputs equal 1, correctly realizing 3-bit AND.

Q32 [2 marks, ~4 min]. Biological Neuron $\rightarrow$ ANN Mapping

Give the artificial-neuron analogue of: (a) synapse, (b) action potential (firing), (c) dendrite.

(a) Synapse $\rightarrow$ weight w_i (connection strength). (b) Action potential/firing $\rightarrow$ the activation function's thresholding decision. (c) Dendrite $\rightarrow$ input x_i (receives the incoming signal).

Q33 [3 marks, ~6 min]. MP Neuron: 3-Input Majority (Numeric)

Design an MP neuron that outputs 1 whenever at least 2 of 3 binary inputs are 1.

Solution: use three excitatory inputs, each of weight 1, with threshold $\theta = 2$:

$$y = 1 \iff x_1 + x_2 + x_3 \geq 2$$

Answer: this is 1 exactly when 2 or 3 inputs are active, correctly realizing the majority function.

Q34 [3 marks, ~5 min]. Steepest Descent Direction

Prove that $-\nabla f(w)$ is the direction of steepest decrease of f at w .

Given: f differentiable at w , and a unit vector $\mathbf{d}$ specifying a direction of travel.

To Prove: among all unit vectors $\mathbf{d}$, f decreases fastest when $\mathbf{d} = -\nabla f(w)/\|\nabla f(w)\|$.

Proof: by the first-order Taylor expansion,

$$f(w + \eta \mathbf{d}) \approx f(w) + \eta \nabla f(w) \cdot \mathbf{d}$$

Minimizing $f(w + \eta \mathbf{d})$ is equivalent to minimizing $\nabla f(w) \cdot \mathbf{d}$. By the Cauchy–Schwarz inequality,

$$\nabla f(w) \cdot \mathbf{d} \geq -\|\nabla f(w)\| \|\mathbf{d}\| = -\|\nabla f(w)\|$$

with equality exactly when $\mathbf{d} = -\nabla f(w)/\|\nabla f(w)\|$. Substituting this direction:

$$f(w - \eta \nabla f(w)) \approx f(w) - \eta \|\nabla f(w)\|^2 < f(w)$$

for $\nabla f(w) \neq 0$ and η sufficiently small, confirming this is the direction of fastest decrease.

Hence Proved. ■

Q35 [6 marks, ~9 min]. Gradient Descent Finite-Step Convergence

Prove that gradient descent on an L -smooth function bounded below reaches $\|\nabla f\| \leq \varepsilon$ within a finite, explicit number of steps.

Given: f is L -smooth ($\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|$ for all x, y) and bounded below by f^* .

To Prove: for any $\varepsilon > 0$, there exists a finite, explicitly computable T such that

$$\min_{0 \leq t < T} \|\nabla f(w_t)\| \leq \varepsilon$$

Proof.

Step 1 (Descent Lemma). For any x, y , by the fundamental theorem of calculus followed by Cauchy–Schwarz and L -Lipschitzness of ∇f :

$$f(y) \leq f(x) + \nabla f(x) \cdot (y - x) + \frac{L}{2} \|y - x\|^2$$

Step 2. Applying this with $y = w_{t+1} = w_t - \eta \nabla f(w_t)$ and choosing $\eta = 1/L$:

$$f(w_{t+1}) \leq f(w_t) - \frac{1}{2L} \|\nabla f(w_t)\|^2$$

Step 3 (telescoping over T steps). Summing for $t = 0, \dots, T - 1$ and using $f(w_T) \geq f^*$:

$$\sum_{t=0}^{T-1} \|\nabla f(w_t)\|^2 \leq 2L(f(w_0) - f^*)$$

Step 4. Since the minimum of T non-negative terms is at most their average:

$$\min_{0 \leq t < T} \|\nabla f(w_t)\|^2 \leq \frac{2L(f(w_0) - f^*)}{T}$$

Setting

$$T = \left\lceil \frac{2L(f(w_0) - f^*)}{\varepsilon^2} \right\rceil$$

makes the right-hand side $\leq \varepsilon^2$, so some iterate among $w_0, \dots, w_{T-1}$ satisfies $\|\nabla f(w_t)\| \leq \varepsilon$.

Hence Proved. ■

Q36 [5 marks, ~8 min]. Inverted Dropout Proof

For inverted dropout with keep-probability p , prove $\mathbb{E}[\text{output}] = x$.

Given: pre-dropout activation h (fixed given the input), keep-mask $m \sim \text{Bernoulli}(p)$, inverted dropout output $h_{\text{drop}} = mh/p$.

To Prove: $\mathbb{E}[h_{\text{drop}}] = h$.

Proof:

$$\mathbb{E}[h_{\text{drop}}] = \mathbb{E}\left[\frac{mh}{p}\right] = \frac{h}{p} \mathbb{E}[m] = \frac{h}{p} \cdot p = h$$

Since the expected training-time output already equals the true value h , no rescaling is required at test time; the network is evaluated as-is.

Hence Proved. ■

Q37 [4 marks, ~6 min]. Batch Norm Gradients

For $z_i = \gamma \hat{x}_i + \beta$ over a batch of size m , given upstream gradients $\delta_i = \partial L / \partial z_i$, derive $\partial L / \partial \gamma$ and $\partial L / \partial \beta$.

Given: $z_i = \gamma \hat{x}_i + \beta$ for $i = 1, \dots, m$, with γ, β shared (broadcast) across the batch.

Derivation: since γ and β each affect every z_i in the batch, their total gradient sums the contribution from each batch element:

$$\frac{\partial L}{\partial \gamma} = \sum_{i=1}^m \frac{\partial L}{\partial z_i} \frac{\partial z_i}{\partial \gamma} = \sum_{i=1}^m \delta_i \hat{x}_i$$

$$\frac{\partial L}{\partial \beta} = \sum_{i=1}^m \frac{\partial L}{\partial z_i} \frac{\partial z_i}{\partial \beta} = \sum_{i=1}^m \delta_i$$

Q38 [4 marks, ~6 min]. Batch Normalization — Explain

Explain what batch normalization computes, how train-time differs from test-time, and why it helps.

Computation. Each pre-activation is normalized over the mini-batch, $\hat{x} = (x - \mu_B) / \sqrt{\sigma_B^2 + \epsilon}$, then transformed by a learnable scale and shift, $z = \gamma \hat{x} + \beta$, allowing the network to undo the normalization if beneficial for a given unit.

Train vs. test. At training time, μ_B and σ_B^2 are computed from the current mini-batch. At test time, a running average of μ_B, σ_B^2 accumulated during training is used instead, since a single test example (or an arbitrary batch) provides no reliable batch statistic.

Benefits. Batch normalization reduces internal covariate shift (each layer sees a more stable input distribution as earlier layers' parameters change during training), permits the use of higher learning rates, and has a mild regularizing effect arising from the per-batch noise in μ_B, σ_B^2 .

Q39 [3 marks, ~6 min]. Eigenvalues/Eigenvectors (Numeric)

Find the eigenvalues and eigenvectors of $A = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix}$.

Given: $A = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix}$.

Solution: the characteristic equation is

$$\det(A - \lambda I) = (4 - \lambda)(3 - \lambda) - 2 = \lambda^2 - 7\lambda + 10 = (\lambda - 5)(\lambda - 2) = 0$$

giving $\lambda_1 = 5, \lambda_2 = 2$.

For $\lambda = 5$: $(A - 5I)v = 0 \Rightarrow \begin{pmatrix} -1 & 1 \\ 2 & -2 \end{pmatrix} v = 0 \Rightarrow v_1 = v_2$, eigenvector $(1, 1)$. For $\lambda = 2$:

$(A - 2I)v = 0 \Rightarrow \begin{pmatrix} 2 & 1 \\ 2 & 1 \end{pmatrix} v = 0 \Rightarrow v_2 = -2v_1$, eigenvector $(1, -2)$.

Answer: $\lambda_1 = 5, v_1 = (1, 1)$ and $\lambda_2 = 2, v_2 = (1, -2)$.

Q40 [5 marks, ~10 min]. RMSProp Two-Step (Numeric)

$w_0 = 0.5, \eta = 0.05, \beta = 0.9, \epsilon \approx 0$. Gradients: $g_1 = 0.6, g_2 = 0.9$. Compute w_1, w_2 .

Given: $w_0 = 0.5, \eta = 0.05, \beta = 0.9, E[g^2]_0 = 0$.

Solution. Step 1: $E[g^2]_1 = 0.9(0) + 0.1(0.36) = 0.036, \sqrt{0.036} = 0.1897$.

$$w_1 = 0.5 - 0.05 \left(\frac{0.6}{0.1897} \right) = 0.5 - 0.15811 = 0.34189$$

Step 2: $E[g^2]_2 = 0.9(0.036) + 0.1(0.81) = 0.1134, \sqrt{0.1134} = 0.3367$.

$$w_2 = 0.34189 - 0.05 \left(\frac{0.9}{0.3367} \right) = 0.34189 - 0.13362 = 0.20827$$

Answer: $w_1 = 0.34189, w_2 = 0.20827$

Q41 [4 marks, ~8 min]. Leaky ReLU Forward/Backward (Numeric)

$\alpha = 0.01$; pre-activations $a_1 = 2.5, a_2 = -1.8$; downstream contribution = 0.4 for each. Compute forward outputs and δ 's; contrast with plain ReLU for a_2 .

Given: Leaky ReLU $\varphi(a) = a$ if $a > 0$, else αa ; $\varphi'(a) = 1$ if $a > 0$, else α .

Solution. Neuron 1 ($a_1 = 2.5 > 0$): output = 2.5; $\delta_1 = 0.4 \times \varphi'(2.5) = 0.4 \times 1 = 0.4$. Neuron 2 ($a_2 = -1.8 < 0$): output = $\alpha a_2 = 0.01(-1.8) = -0.018$; $\delta_2 = 0.4 \times \varphi'(-1.8) = 0.4 \times 0.01 =$

0.004.

Answer: $\delta_1 = 0.4, \delta_2 = 0.004$. Plain ReLU would give $\varphi'(-1.8) = 0$ exactly, so $\delta_2 = 0$ (a fully dead unit); Leaky ReLU keeps a small non-zero gradient flowing.

Q42 [3 marks, ~6 min]. Xavier/He Variance (Numeric)

Layer A: $256 \rightarrow 128$, tanh. Layer B: $128 \rightarrow 64$, ReLU. Compute the prescribed initialization variance for each.

Given: Layer A: $n_{in} = 256, n_{out} = 128$, tanh (Xavier). Layer B: $n_{in} = 128$, ReLU (He).

Solution:

$$\text{Layer A: } \text{Var}(w) = \frac{2}{n_{in} + n_{out}} = \frac{2}{384} = 0.00521$$

$$\text{Layer B: } \text{Var}(w) = \frac{2}{n_{in}} = \frac{2}{128} = 0.015625$$

Answer: $\boxed{\text{Layer A: } 0.00521, \text{ Layer B: } 0.015625}$

Q43 [5 marks, ~10 min]. Batch Norm Forward (Numeric)

$x = (4, 8, 6, 2)$, $\gamma = 2, \beta = 1$. Compute the batch mean, variance, normalized values, and output.

Given: $x = (4, 8, 6, 2)$, $\gamma = 2, \beta = 1$.

Solution:

$$\mu = \frac{4 + 8 + 6 + 2}{4} = 5, \quad \sigma^2 = \frac{(-1)^2 + 3^2 + 1^2 + (-3)^2}{4} = 5, \quad \sigma = \sqrt{5} = 2.2361$$

$$\hat{x} = (-0.4472, 1.3416, 0.4472, -1.3416)$$

$$z = 2\hat{x} + 1$$

Answer: $\boxed{z = (0.1056, 3.6832, 1.8944, -1.6832)}$

Q44 [4 marks, ~8 min]. L2 Regularized Gradient (Numeric)

$w = (0.6, -0.4, 0.9)$, $\nabla L = (0.2, 0.1, -0.3)$, $\lambda = 0.01, \eta = 0.1$. Penalty $\lambda\|w\|^2$. Compute the regularized gradient and updated weight.

Given: $w = (0.6, -0.4, 0.9)$, $\nabla L = (0.2, 0.1, -0.3)$, $\lambda = 0.01, \eta = 0.1$.

Solution. The penalty $\lambda\|w\|^2$ contributes gradient $2\lambda w$:

$$\nabla L_{\text{reg}} = \nabla L + 2\lambda w = (0.2, 0.1, -0.3) + 0.02(0.6, -0.4, 0.9) = (0.212, 0.092, -0.282)$$

$$w_{\text{new}} = w - \eta \nabla L_{\text{reg}} = (0.6, -0.4, 0.9) - 0.1(0.212, 0.092, -0.282)$$

Answer: $\boxed{w_{\text{new}} = (0.5788, -0.4092, 0.9282)}$

Q45 [3 marks, ~6 min]. Inverted Dropout (Numeric)

$h = (2, 5, 3, 8)$, $p = 0.75$, realized mask $m = (1, 0, 1, 1)$. Compute the inverted-dropout output.

Given: $h = (2, 5, 3, 8)$, $p = 0.75$, $m = (1, 0, 1, 1)$.

Solution: $h_{\text{drop},i} = m_i h_i / p$:

$$h_{\text{drop}} = \left(\frac{2}{0.75}, \frac{0}{0.75}, \frac{3}{0.75}, \frac{8}{0.75} \right)$$

Answer: $h_{\text{drop}} = (2.6667, 0, 4.0000, 10.6667)$

Q46 [4 marks, ~8 min]. Bias-Variance from Predictions (Numeric)

$\hat{f} = (3.2, 2.8, 3.5, 2.5)$ from 4 bootstrap samples at a fixed test point, $f(x) = 3.0$, noise-free. Compute the empirical bias, variance, and expected squared error.

Given: $\hat{f}_1 = 3.2, \hat{f}_2 = 2.8, \hat{f}_3 = 3.5, \hat{f}_4 = 2.5$; $f(x) = 3.0$; $\sigma^2 = 0$.

Solution. Mean prediction:

$$\bar{f} = \frac{3.2 + 2.8 + 3.5 + 2.5}{4} = 3.0$$

$$\text{Bias} = \bar{f} - f(x) = 3.0 - 3.0 = 0$$

$$\text{Variance} = \frac{(0.2)^2 + (-0.2)^2 + (0.5)^2 + (-0.5)^2}{4} = \frac{0.58}{4} = 0.145$$

Answer: $\text{Bias} = 0, \text{Variance} = 0.145, \text{Expected squared error} = 0 + 0.145 + 0 = 0.145$

Q47 [4 marks, ~8 min]. GD Closed-Form Convergence (Numeric)

$f(x) = (x - 3)^2$, $\eta = 0.2$, $x_0 = 0$. Derive the closed form for x_t and compute $x_1, \dots, x_4$.

Given: $f(x) = (x - 3)^2$, $\eta = 0.2$, $x_0 = 0$.

Solution. $f'(x) = 2(x - 3)$, so the update rule is

$$x_{t+1} = x_t - 0.2 \cdot 2(x_t - 3) = 0.6x_t + 1.2$$

Substituting $u_t = x_t - 3$: $u_{t+1} = 0.6u_t$, so $u_t = 0.6^t u_0 = -3(0.6)^t$, giving

$$x_t = 3 - 3(0.6)^t$$

$$x_1 = 1.2, \quad x_2 = 1.92, \quad x_3 = 2.352, \quad x_4 = 2.6112$$

Answer: $x_t = 3 - 3(0.6)^t$; as $t \rightarrow \infty$, $x_t \rightarrow 3$, the true minimum.